Process A:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <errno.h>
#include <signal.h>

#define PIPE_A_TO_B "/tmp/ipc_chat_A_to_B"
#define PIPE_B_TO_A "/tmp/ipc_chat_B_to_A"
#define SIZE 256

int ensure_fifo(const char *path) {
    struct stat st;

    if (mkfifo(path, 0600) == 0)
        return 0;

    if (errno != EEXIST) {
        perror("mkfifo");
        return -1;
    }

    if (lstat(path, &st) == -1) {
        perror("lstat");
        return -1;
```

```
    }

    if (!S_ISFIFO(st.st_mode)) {
        fprintf(stderr, "%s is not a FIFO\n", path);
        return -1;
    }

    return 0;
}

int main(void) {
    int write_fd, read_fd;
    char message[SIZE];
    char reply[SIZE];

    signal(SIGPIPE, SIG_IGN);

    if (ensure_fifo(PIPE_A_TO_B) == -1 ||
        ensure_fifo(PIPE_B_TO_A) == -1)
        return EXIT_FAILURE;

    printf("=== IPC CHAT: PROCESS A ===\n");
    printf("Waiting for Process B...\n");

    write_fd = open(PIPE_A_TO_B, O_WRONLY);
    if (write_fd == -1) {
        perror("Opening A-to-B pipe");
        return EXIT_FAILURE;
    }
```

```
read_fd = open(PIPE_B_TO_A, O_RDONLY);
if (read_fd == -1) {
    perror("Opening B-to-A pipe");
    close(write_fd);
    return EXIT_FAILURE;
}

FILE *out = fdopen(write_fd, "w");
FILE *in = fdopen(read_fd, "r");

if (out == NULL || in == NULL) {
    perror("fdopen");

    if (out != NULL)
        fclose(out);
    else
        close(write_fd);

    if (in != NULL)
        fclose(in);
    else
        close(read_fd);

    return EXIT_FAILURE;
}

printf("Connected! Type 'exit' to quit.\n\n");
```

```
while (1) {
    printf("Process A - Enter message: ");
    fflush(stdout);

    if (fgets(message, SIZE, stdin) == NULL)
        break;

    if (strchr(message, '\n') == NULL && !feof(stdin)) {
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;
        printf("Message too long. Try again.\n");
        continue;
    }

    message[strcspn(message, "\n")] = '\0';

    if (fprintf(out, "%s\n", message) < 0 ||
        fflush(out) == EOF) {
        perror("Sending message");
        break;
    }

    if (strcmp(message, "exit") == 0)
        break;

    if (fgets(reply, SIZE, in) == NULL) {
        printf("Process B disconnected.\n");
        break;
```

```
        }

        reply[strcspn(reply, "\n")] = '\0';

        printf("Process B: %s\n\n", reply);

        if (strcmp(reply, "exit") == 0)
            break;
    }

    fclose(out);
    fclose(in);

    printf("Process A disconnected.\n");
    return EXIT_SUCCESS;
}
```

Process B:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <errno.h>
#include <signal.h>

#define PIPE_A_TO_B "/tmp/ipc_chat_A_to_B"
#define PIPE_B_TO_A "/tmp/ipc_chat_B_to_A"
```

```
#define SIZE 256

int ensure_fifo(const char *path) {
    struct stat st;

    if (mkfifo(path, 0600) == 0)
        return 0;

    if (errno != EEXIST) {
        perror("mkfifo");
        return -1;
    }

    if (lstat(path, &st) == -1) {
        perror("lstat");
        return -1;
    }

    if (!S_ISFIFO(st.st_mode)) {
        fprintf(stderr, "%s is not a FIFO\n", path);
        return -1;
    }

    return 0;
}

int main(void) {
    int read_fd, write_fd;
    char message[SIZE];
```

```
char reply[SIZE];

signal(SIGPIPE, SIG_IGN);

if (ensure_fifo(PIPE_A_TO_B) == -1 ||
    ensure_fifo(PIPE_B_TO_A) == -1)
    return EXIT_FAILURE;

printf("=== IPC CHAT: PROCESS B ===\n");
printf("Waiting for Process A...\n");

read_fd = open(PIPE_A_TO_B, O_RDONLY);
if (read_fd == -1) {
    perror("Opening A-to-B pipe");
    return EXIT_FAILURE;
}

write_fd = open(PIPE_B_TO_A, O_WRONLY);
if (write_fd == -1) {
    perror("Opening B-to-A pipe");
    close(read_fd);
    return EXIT_FAILURE;
}

FILE *in = fdopen(read_fd, "r");
FILE *out = fdopen(write_fd, "w");

if (in == NULL || out == NULL) {
    perror("fdopen");
```

```
    if (in != NULL)
        fclose(in);
    else
        close(read_fd);

    if (out != NULL)
        fclose(out);
    else
        close(write_fd);

    return EXIT_FAILURE;
}

printf("Connected! Type 'exit' to quit.\n\n");

while (1) {
    if (fgets(message, SIZE, in) == NULL) {
        printf("Process A disconnected.\n");
        break;
    }

    if (strchr(message, '\n') == NULL && !feof(in)) {
        fprintf(stderr, "Received message too long.\n");
        break;
    }

    message[strcspn(message, "\n")] = '\0';
```

```
printf("Process A: %s\n", message);

if (strcmp(message, "exit") == 0)
    break;

printf("Process B - Enter reply: ");
fflush(stdout);

if (fgets(reply, SIZE, stdin) == NULL)
    break;

if (strchr(reply, '\n') == NULL && !feof(stdin)) {
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF)
        ;
    printf("Message too long. Try again.\n");
    continue;
}

reply[strcspn(reply, "\n")] = '\0';

if (fprintf(out, "%s\n", reply) < 0 ||
    fflush(out) == EOF) {
    perror("Sending reply");
    break;
}

if (strcmp(reply, "exit") == 0)
    break;
```

```
        printf("\n");
    }

    fclose(in);
    fclose(out);

    printf("Process B disconnected.\n");
    return EXIT_SUCCESS;
}
```